

PRESENTATION STUDY GUIDE: RTK Positioning Simulation for UAV Emergency Rescue

1. Document Overview and Executive Instructions

Meta-Instruction for the Presenter:

This guide is structured for a 4-slide technical presentation. The narrative and data provided are synthesized from verified Level 1 (Standalone ROS 2) and Level 2 (PX4/Gazebo Integrated) simulation logs. As the presenter, you must emphasize that all performance metrics—specifically the 96.7% accuracy improvement—are derived from empirical results captured in these high-fidelity simulation environments.

2. SLIDE 1 — RTK Positioning: The Precision Backbone of UAV Rescue

2.1. Script: "What to Say"

"Standard GPS typically provides a positioning accuracy of 1 to 5 meters. While sufficient for general navigation, this margin of error is unacceptable for precision rescue operations, where a UAV must execute a fixed-point landing on a rescue target or navigate through tight debris fields.

To bridge this gap, we utilize Real-Time Kinematic positioning, or RTK. Unlike standard GNSS which relies on code-phase measurements, RTK utilizes a Base-Rover relationship to resolve carrier-phase ambiguities. By processing RTCM correction data from a base station at a known location, the rover can eliminate atmospheric delays and satellite clock errors in real-time.

Our module manages the positioning logic across four specific fix states, publishing high-precision coordinates to the `/uav/rtk_position` ROS 2 topic:"

Status	Description	Typical Error
GNSS_ONLY	No correction applied	1–5 m
RTK_FLOAT	Correction received; ambiguity unresolved	0.2–0.5 m
RTK_FIXED	Full fix; carrier-phase ambiguity resolved	0.02–0.05 m

Status	Description	Typical Error
CORRECTION_LOST	Correction signal interrupted	1–5 m

2.2. Delivery: "How to Say It"

- **Pacing:** Use a deliberate, slow pace when contrasting "meters" versus "centimeters" to ensure the scale of precision is felt by the audience.
- **Body Language:** Incorporate cues for eye contact across the room. Use a "pinching" hand gesture when discussing "centimeter-level" to visually reinforce the concept of precision.
- **Tone:** Maintain a confident, non-rushed tone, particularly while explaining the technical distinction of carrier-phase ambiguity.

2.3. Transition Statement

"To validate the efficacy of this logic before hardware deployment, we implemented a rigorous two-phase simulation architecture."

3. SLIDE 2 — Two-Phase Simulation Architecture

3.1. Script: "What to Say"

"Our architecture is divided into two distinct levels to isolate control logic from environmental complexity.

Level 1 is a standalone ROS 2 environment. It utilizes four core nodes—the `simulated_uav_node`, `base_station_node`, `rtk_positioning_node`, and the `rtk_status_manager`. We tested a 50m x 50m square flight path at a target velocity of 5m/s and an altitude of 30 meters. This phase verifies our coordinate transformations and produces an 18-column CSV log for baseline analysis.

Level 2 integrates the module with the PX4 autopilot and Gazebo Harmonic. We source real UAV pose data using the `ros_gz_bridge`.

A key technical decision was choosing `ros_gz_bridge` to access the Gazebo navsat sensor directly, rather than relying on the `px4_msgs` package. This provided a cleaner, lightweight solution for our environment. More importantly, it allowed us to ingest raw, unfiltered sensor data, which is more appropriate for proving our noise modeling and RTK logic than using the EKF-filtered odometry output from the autopilot. Level 2 generates 22-column logs and 6 analysis graphs, including cross-validation against QGroundControl ULogs."

3.2. Delivery: "How to Say It"

- **Technique:** Use a 'counting-on-fingers' technique when naming the four nodes in Level 1 to emphasize the modularity of the design.
- **Emphasis:** Deliberately slow down during the 'Key Technical Decision' section. This demonstrates engineering ownership and a deep understanding of the trade-offs between raw sensor data and filtered state estimates.

3.3. Transition Statement

"Having verified the architecture, we can now examine the empirical evidence that justifies this module's inclusion in the rescue stack."

4. SLIDE 3 — Results, Validation, and Honest Limitations

4.1. Script: "What to Say"

"The quantitative results demonstrate a significant leap in operational capability. In our Level 2 mission—conducted in the ETH Zurich world at 50m altitude and a flight speed of 17.8 km/h—we compared three distinct positioning tiers.

Raw GNSS error averaged 2.394 meters. The standard PX4 GPS EPH reported an uncertainty of 0.900 meters. However, our **RTK_FIXED** state achieved a mean error of only 0.048 meters with a standard deviation of 0.012 meters. This represents a **96.7% accuracy improvement** over raw GNSS.

We also validated system robustness by injecting a correction signal loss at $t=45s$. The system successfully transitioned to **CORRECTION_LOST**, then recovered the **RTK_FIXED** state within 5 seconds of the signal returning at $t=50s$.

In the interest of engineering maturity, we acknowledge current limitations: the simulation uses Gaussian noise models rather than complex signal propagation, we do not yet perform binary RTCM decoding, and the results are currently limited to a software-only environment."

4.2. Delivery: "How to Say It"

- **Pauses:** Direct a full two-second pause after stating "96.7%" to allow the magnitude of the improvement to resonate.
- **Tone:** Maintain a matter-of-fact, non-defensive tone when listing limitations. This builds significant professional credibility by showing you understand the gap between simulation and field hardware.

4.3. Transition Statement

"These results provide the empirical foundation for our system-wide integration and future hardware roadmap."

5. SLIDE 4 — Integration, Challenges, and Next Steps

5.1. Script: "What to Say"

"The RTK module serves as the critical third link in our five-module rescue system. The data flow begins with a BeiDou Short Message, moves through QGC Control, and reaches our RTK Positioning module via the `/uav/rtk_position` topic. This high-precision coordinate is then ingested by Path Planning and Target Detection to ensure mission success.

Development was not without obstacles. We bypassed WSL2 rendering limitations by utilizing headless mode and resolved the `px4_msgs` dependency through the bridge strategy mentioned earlier. We also utilized real QGC missions to ensure our movement data reflected realistic flight dynamics.

Our roadmap targets native Ubuntu 24.04 deployment and integration with physical u-blox hardware. Crucially, the interface is hardware-agnostic by design; whether the input is a Gazebo sensor or a physical receiver, the core logic remains identical."

5.2. Delivery: "How to Say It"

- **Visual Aid:** Point specifically to the RTK module's position in the system flow if a diagram is displayed.
 - **Fluency:** Speak ROS topic names like `/uav/rtk_position` and coordinate conventions like 'WGS84 to ENU' clearly and with technical authority.
-

6. FULL Q&A SECTION: ANTICIPATED QUESTIONS AND MODEL ANSWERS

Q: Why was Gaussian noise used for the RTK model?

A: For these simulation levels, Gaussian noise serves as a mathematically rigorous proxy for signal interference. It allows us to demonstrate the module's ability to handle varying precision levels—from 1.5m down to 0.03m—and validate the state machine transitions without the prohibitive overhead of simulating full atmospheric signal propagation.

Q: What is the primary difference between Level 1 and Level 2?

A: Level 1 is a logic-validation environment using a synthetic 5m/s path and time-based status transitions. Level 2 is 'MAVLink-aware,' sourcing real UAV pose data from the Gazebo environment and triggering RTK status transitions based on the availability and quality of a simulated RTCM correction stream.

Q: What ROS 2 messages are used for the interface?

A: We utilize standard ROS 2 interface contracts: `sensor_msgs/NavSatFix` for global positioning, `geometry_msgs/PoseStamped` for local coordinates, and `std_msgs/Header` to maintain strict temporal synchronization across the rescue stack.

Q: How does RTK accuracy specifically impact the rescue mission?

A: Precision is the difference between a successful aid delivery and a mission failure. A 1–5m GNSS error could cause a UAV to miss a landing pad or fail to resolve a survivor in the camera's FOV. An RTK_FIXED error of 0.048m enables the UAV to land exactly where needed to deliver life-saving supplies.

Q: How resilient is the system to signal loss?

A: Very resilient. The simulation includes a `CORRECTION_LOST` state where the error model expands to a 2.5m standard deviation to simulate stale data. The logic is designed to re-acquire an `RTK_FIXED` state within 5 seconds of the correction stream being restored, as demonstrated in our robustness tests.

Q: Why was the `ros_gz_bridge` preferred over the standard PX4 DDS bridge?

A: This was a strategic trade-off. The target system lacked the `px4_msgs` package, and the `ros_gz_bridge` offered a lightweight alternative. Furthermore, the Gazebo navsat sensor provides raw GPS data, which is a more appropriate input for testing an RTK positioning module than the EKF-filtered odometry provided by the autopilot.

Q: Which coordinate systems were used during testing?

A: The design is coordinate-agnostic. For Level 1, we configured the `base_station.yaml` with Beijing-area coordinates. For Level 2, we used the PX4 default ETH Zurich origin. In both scenarios, the module successfully converted WGS84 global coordinates into East-North-Up (ENU) local coordinates relative to the base station, adhering to standard ROS 2 navigation conventions.